Practical : 4	Game-Playing AI Agent Using Minimax and Alpha-Beta Pruning.
Date : 17/08/26	

Aim: -

To develop a game-playing AI agent for Tic-Tac-Toe using the Minimax algorithm and improve its performance using Alpha-Beta Pruning.

Algorithm:-

1. Start the program.
2. Create an empty Tic-Tac-Toe board.
3. Display the board positions from 1 to 9.
4. Allow the user to select either Minimax or Alpha-Beta Pruning.
5. The human player enters a valid position and plays as O.
6. Check whether the human player has won or the game is a draw.
7. The AI analyzes the possible moves.
8. Use the selected algorithm to find the best move for the AI.
9. The AI plays as X.
10. Check whether the AI has won or the game is a draw.
11. Repeat the process until either the human wins, the AI wins, or the game ends in a draw.
12. Display the final board and stop the program.

Program:-

```
# ============================================================
# PRACTICAL 4
# Game Playing Agent using Minimax and Alpha-Beta Pruning
# Real-Life Example: Tic-Tac-Toe
# ============================================================

import math

# Create Tic-Tac-Toe Board
board = [
    " ", " ", " ",
    " ", " ", " ",
    " ", " ", " "
]


# Display the Board
def display_board():
    print()
    print("---------")
    print("|", board[0], "|", board[1], "|", board[2], "|")
    print("---------")
    print("|", board[3], "|", board[4], "|", board[5], "|")
    print("---------")
    print("|", board[6], "|", board[7], "|", board[8], "|")
    print("---------")
    print()


# Check Whether a Player Has Won
def check_winner(player):

    winning_positions = [
        (0, 1, 2),
        (3, 4, 5),
        (6, 7, 8),
        (0, 3, 6),
        (1, 4, 7),
        (2, 5, 8),
```

```
        (0, 4, 8),
        (2, 4, 6)
    ]

    for position in winning_positions:
        if (
            board[position[0]] == player and
            board[position[1]] == player and
            board[position[2]] == player
        ):
            return True

    return False


# Check Whether Board is Full
def is_board_full():
    return " " not in board


# Minimax Algorithm
def minimax(is_maximizing):

    # Terminal States
    if check_winner("X"):
        return 1

    if check_winner("O"):
        return -1

    if is_board_full():
        return 0

    # Maximizing Player - AI
    if is_maximizing:

        best_score = -math.inf

        for i in range(9):
            if board[i] == " ":

                board[i] = "X"
```

```
            score = minimax(False)
            board[i] = " "

            best_score = max(best_score, score)

        return best_score

    # Minimizing Player - Human
    else:

        best_score = math.inf

        for i in range(9):
            if board[i] == " ":

                board[i] = "O"
                score = minimax(True)
                board[i] = " "

                best_score = min(best_score, score)

        return best_score


# Find Best Move Using Minimax
def find_best_move():

    best_score = -math.inf
    best_move = None

    for i in range(9):
        if board[i] == " ":

            board[i] = "X"
            score = minimax(False)
            board[i] = " "

            if score > best_score:
                best_score = score
                best_move = i

    return best_move
```

```
# Alpha-Beta Pruning Algorithm
def alpha_beta(is_maximizing, alpha, beta):

    # Terminal States
    if check_winner("X"):
        return 1

    if check_winner("O"):
        return -1

    if is_board_full():
        return 0

    # Maximizing Player - AI
    if is_maximizing:

        best_score = -math.inf

        for i in range(9):
            if board[i] == " ":

                board[i] = "X"

                score = alpha_beta(
                    False,
                    alpha,
                    beta
                )

                board[i] = " "

                best_score = max(
                    best_score,
                    score
                )

                alpha = max(
                    alpha,
                    best_score
                )
```

```
            # Alpha-Beta Pruning
            if beta <= alpha:
                break

    return best_score

# Minimizing Player - Human
else:

    best_score = math.inf

    for i in range(9):
        if board[i] == " ":

            board[i] = "O"

            score = alpha_beta(
                True,
                alpha,
                beta
            )

            board[i] = " "

            best_score = min(
                best_score,
                score
            )

            beta = min(
                beta,
                best_score
            )

            # Alpha-Beta Pruning
            if beta <= alpha:
                break

    return best_score
```

```
# Find Best Move Using Alpha-Beta Pruning
def find_best_move_alpha_beta():

    best_score = -math.inf
    best_move = None

    alpha = -math.inf
    beta = math.inf

    for i in range(9):
        if board[i] == " ":

            board[i] = "X"

            score = alpha_beta(
                False,
                alpha,
                beta
            )

            board[i] = " "

            if score > best_score:
                best_score = score
                best_move = i

    return best_move


# ============================================================
# Main Game
# ============================================================

print("=" * 50)
print(" TIC-TAC-TOE AI AGENT")
print("=" * 50)

print("\nYou are O")
print("AI is X\n")

# Display Position Numbers
print("Board Positions:\n")
```

```
print(" 1 | 2 | 3 ")
print("-------")
print(" 4 | 5 | 6 ")
print("-------")
print(" 7 | 8 | 9 ")

print("\nSelect AI Algorithm")
print("1. Minimax")
print("2. Alpha-Beta Pruning")

choice = input("\nEnter your choice (1 or 2): ")


# Game Loop
while True:

    # Human Move
    display_board()

    try:
        position = int(
            input("Enter your position (1-9): ")
        )

    except ValueError:
        print("Please enter a number.")
        continue

    position = position - 1

    # Check valid position
    if position < 0 or position > 8:
        print("Invalid position.")
        continue

    # Check whether position is empty
    if board[position] != " ":
        print("Position already occupied.")
        continue

    # Place Human Move
```

```
board[position] = "O"

# Check Human Winner
if check_winner("O"):
    display_board()
    print("Congratulations!")
    print("You Win!")
    break

# Check Draw
if is_board_full():
    display_board()
    print("Game Draw!")
    break

# AI Move
print("\nAI is thinking...")

if choice == "1":
    ai_move = find_best_move()

elif choice == "2":
    ai_move = find_best_move_alpha_beta()

else:
    print("Invalid choice. Using Minimax.")
    ai_move = find_best_move()

# Make AI Move
board[ai_move] = "X"

print("AI selected position:", ai_move + 1)

# Check AI Winner
if check_winner("X"):
    display_board()
    print("AI Wins!")
    break

# Check Draw
if is_board_full():
    display_board()
```

```
        print("Game Draw!")
        break


# Final Board
print()
print("=" * 50)
print(" FINAL BOARD")
print("=" * 50)

display_board()

print("Game Completed.")
```

Output:-

```
C:\Users\Vishnu\OneDrive\AI_Practical_4>python practical4_tictactoe.py
==================================================
 TIC-TAC-TOE AI AGENT
==================================================

You are O
AI is X

Board Positions:

 1 | 2 | 3
-----------
 4 | 5 | 6
-----------
 7 | 8 | 9

Select AI Algorithm
1. Minimax
2. Alpha-Beta Pruning

Enter your choice (1 or 2): 1

-------------
|   |   |   |
-------------
|   |   |   |
-------------
|   |   |   |
-------------

Enter your position (1-9): 3

AI is thinking...
AI selected position: 5

-------------
|   |   | O |
-------------
|   | X |   |
-------------
|   |   |   |
-------------

Enter your position (1-9): 7

AI is thinking...
AI selected position: 2

-------------
|   | X | O |
-------------
|   | X |   |
-------------
| O |   |   |
-------------

Enter your position (1-9): 8
```

```
AI is thinking...
AI selected position: 9

-------------
|   | X | O |
-------------
|   | X |   |
-------------
| O | O | X |
-------------

Enter your position (1-9): 1

AI is thinking...
AI selected position: 4

-------------
| O | X | O |
-------------
| X | X |   |
-------------
| O | O | X |
-------------

Enter your position (1-9): 6

-------------
| O | X | O |
-------------
| X | X | O |
-------------
| O | O | X |
-------------

Game Draw!

================================================
 FINAL BOARD
================================================

-------------
| O | X | O |
-------------
| X | X | O |
-------------
| O | O | X |
-------------

Game Completed.
```

Conclusion:-

Thus, a **Game-Playing AI Agent for Tic-Tac-Toe was successfully implemented using Python**. The AI uses the **Minimax algorithm** to analyze possible moves and select the best move. **Alpha-Beta Pruning** is also implemented to improve the search process by reducing unnecessary calculations.